Red Black Tree

Introduction:

A red-black tree is a kind of self-balancing binary search tree where each node has an extra bit, and that bit is often interpreted as the color (red or black). These colors are used to ensure that the tree remains balanced during insertions and deletions. Although the balance of the tree is not perfect, it is good enough to reduce the searching time and maintain it around O(log n) time, where n is the total number of elements in the tree. This tree was invented in 1972 by Rudolf Bayer.

It must be noted that as each node requires only 1 bit of space to store the color information, these types of trees show identical memory footprints to the classic (uncolored) binary search tree.

Rules That Every Red-Black Tree Follows:

1. Every node has a color either red or black.
2. The root of the tree is always black.
3. There are no two adjacent red nodes (A red node cannot have a red parent or red child).
4. Every path from a node (including root) to any of its descendants NULL nodes has the same number of black nodes.
5. All leaf nodes are black nodes.

Why Red-Black Trees?

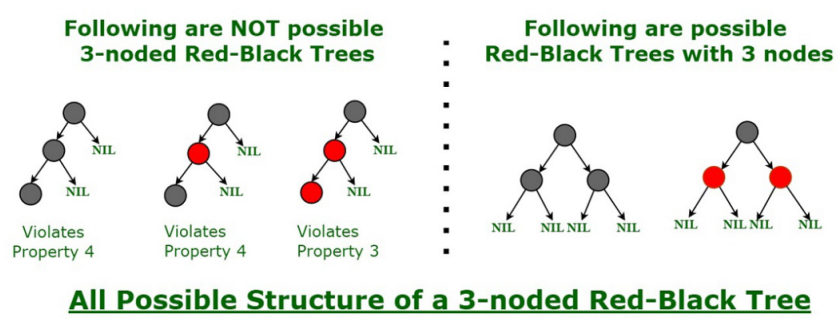
Most of the BST operations (e.g., search, max, min, insert, delete.. etc) take O(h) time where h is the height of the BST. The cost of these operations may become O(n) for a skewed Binary tree. If we make sure that the height of the tree remains O(log n) after every insertion and deletion, then we can guarantee an upper bound of O(log n) for all these operations. The height of a Red-Black tree is always O(log n) where n is the number of nodes in the tree.

Sr. No.	Algorithm	Time Complexity
1.	Search	O(log n)
2.	Insert	O(log n)
3.	Delete	O(log n)

"n" is the total number of elements in the red-black tree.

Comparison with AVL Tree:
The AVL trees are more balanced compared to Red-Black Trees, but they may cause more rotations during insertion and deletion. So if your application involves frequent insertions and deletions, then Red-Black trees should be preferred. And if the insertions and deletions are less frequent and search is a more frequent operation, then AVL tree should be preferred over the Red-Black Tree.

How does a Red-Black Tree ensure balance?
A simple example to understand balancing is, that a chain of 3 nodes is not possible in the Red-Black tree. We can try any combination of colors and see if all of them violate the Red-Black tree property.



Proper structure of three noded Red-black tree

Interesting points about Red-Black Tree:

1. The black height of the red-black tree is the number of black nodes on a path from the root node to a leaf node. Leaf nodes are also counted as black nodes. So, a red-black tree of height h has black height >= h/2.
2. Height of a red-black tree with n nodes is h<= 2 $\log_2(n + 1)$.
3. All leaves (NIL) are black.
4. The black depth of a node is defined as the number of black nodes from the root to that node i.e the number of black ancestors.
5. Every red-black tree is a special case of a binary tree.

Black Height of a Red-Black Tree :

Black height is the number of black nodes on a path from the root to a leaf. Leaf nodes are also counted black nodes. From the above properties 3 and 4, we can derive, **a Red-Black Tree of height h has black-height >= h/2**.

Number of nodes from a node to its farthest descendant leaf is no more than twice as the number of nodes to the nearest descendant leaf.

nearest descendant leaf.

Every Red Black Tree with n nodes has height <= $2Log_2(n+1)$
This can be proved using the following facts:

1. For a general Binary Tree, let **k** be the minimum number of nodes on all root to NULL paths, then n >= $2^k - 1$ (Ex. If k is 3, then n is at least 7). This expression can also be written as k <= $Log_2(n+1)$.
2. From property 4 of Red-Black trees and above claim, we can say in a Red-Black Tree with n nodes, there is a root to leaf path with at-most $Log_2(n+1)$ black nodes.
3. From properties 3 and 5 of Red-Black trees, we can claim that the number of black nodes in a Red-Black tree is at least ⌊ n/2 ⌋ where n is the total number of nodes.

From the above points, we can conclude the fact that Red Black Tree with **n** nodes has a height <= $2Log_2(n+1)$

Search Operation in Red-black Tree:

As every red-black tree is a special case of a binary tree so the searching algorithm of a red-black tree is similar to that of a binary tree.

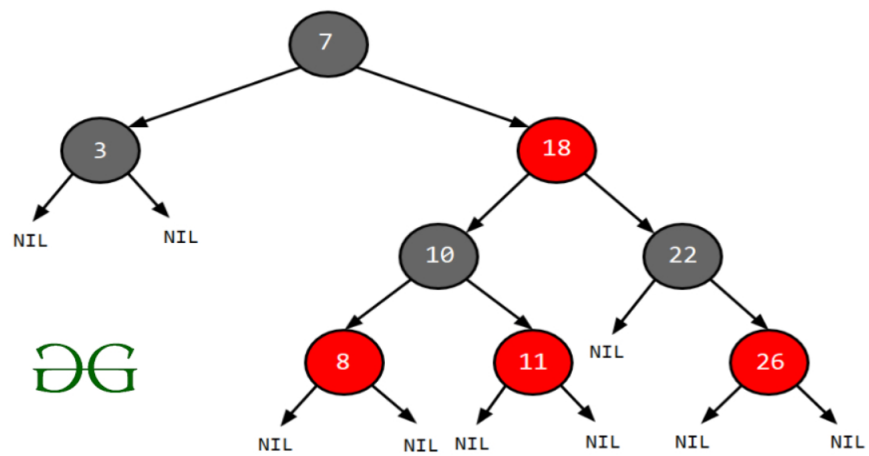
Algorithm:

```
searchElement (tree, val)
Step 1:
If tree -> data = val OR tree = NULL
    Return tree
Else
If val < data
        Return searchElement (tree -> left, val)
    Else
        Return searchElement (tree -> right, val)
    [ End of if ]
[ End of if ]

Step 2: END
```

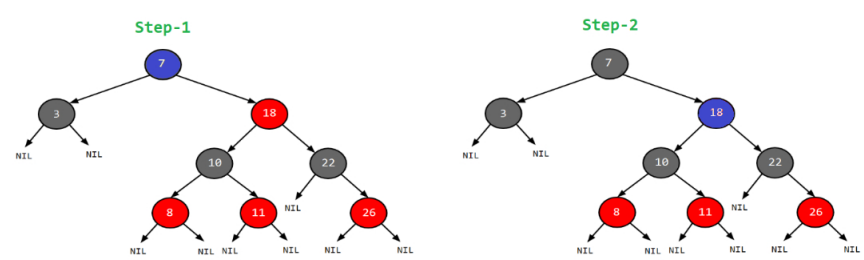
For the program, you can refer it for **AVL tree**.

Example: Searching 11 in the following red-black tree.



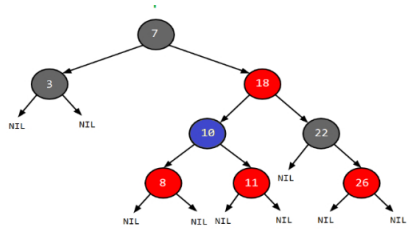
Solution:

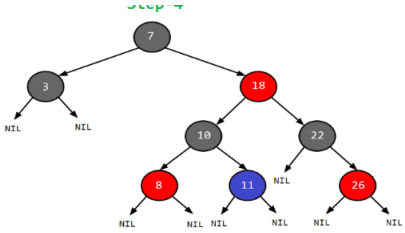
1. Start from the root.
2. Compare the inserting element with root, if less than root, then recurse for left, else recurse for right.
3. If the element to search is found anywhere, return true, else return false.



Step-3 Step-4

Deletion in a Binary Search Tree
Last Updated: 2022-07-01
Floor in BST
Last Updated: 2022-12-29
Ceil in BST
Last Updated: 2022-12-29
Self Balancing BST
Last Updated: 2024-05-01
Introduction to AVL Tree
Last Updated: 2024-05-01
Red Black Tree
Last Updated: 2024-05-01
Applications of BST
Last Updated: 2022-12-18
Find Kth Smallest in BST





Just follow the blue bubble.

In this post, we introduced Red-Black trees and discussed how balance is ensured. The hard part is to maintain balance when keys are added and removed. We have also seen how to search an element from the red-black tree. We will soon be discussing insertion and deletion operations in coming posts on the Red-Black tree.

Applications:

1. Most of the self-balancing BST library functions like map, multiset, and multimap in C++ (or java pacakages like java.util.TreeMap and java.util.TreeSet) use Red-Black Trees.
2. It is used to implement CPU Scheduling Linux. Completely Fair Scheduler uses it.
3. It is also used in the K-mean clustering algorithm in machine learning for reducing time complexity.
4. Moreover, MySQL also uses the Red-Black tree for indexes on tables in order to reduce the searching and insertion time.

Marked as Read

Report An Issue
If you are facing any issue on this page. Please let us know.